FE 520 Introduction to Python for Financial Applications
Assignment 2 100 Points

Rules for Submission -

1. You are required to submit .ipynb or .py file along with .pdf file as part of the submission.
2. Mention your name on your submission. For instance, First Name_Last Name_Assignment 2.ipynb
3. There is a bonus question in 2nd question, make sure to attempt it as well.

All the best !

Question 1 — 20 Points

Dataset -

```
news_block = """
MSFT:
Microsoft's quarterly earnings BEAT expectations, stock surges.
---
AAPL:
Apple announces major UPGRADE to its iPhone lineup, analysts are bullish.
---
GOOG:
Google faces potential DOWNGRADE amid regulatory concerns.
---
META:
Meta platforms MISS revenue targets, stock falls sharply.
---
NVDA:
NVIDIA's outlook strong, analysts reiterate BUY rating.
"""
```

1. Clean the news_block text by removing leading/trailing whitespace and splitting it into a list of individual news items using "---" as the delimiter. Ensure empty items are removed. (4 Points)
2. Write a function called classify_sentiment(snippet) that takes a single news snippet string as input. The function should return "Bullish" if the snippet contains "BEAT", "UPGRADE", or "BUY". It should return "Bearish" if it contains "MISS" or "DOWNGRADE". Otherwise, it should return "Neutral". (4 Points)
3. Using a for loop, iterate through your list of cleaned snippets. For each snippet, extract the stock ticker (the first word before the colon) and use your classify_sentiment function to determine its sentiment. Store the results as a list of tuples, for example: [('MSFT', 'Bullish'), ('AAPL', 'Bullish'), ...]. (4 Points)
4. Calculate an overall sentiment score from your list of tuples. Assign +1 for each "Bullish" snippet and -1 for each "Bearish" snippet. "Neutral" snippets get a score of 0. (4 Points)
5. Create a final, multi-line f-string that reports the total number of snippets analyzed, the count of Bullish and Bearish snippets, and the final overall sentiment score. (4 Points)

Question 2 — 20 Points

Your task is to analyze the historical performance of Tesla Inc. (TSLA) by fetching its data and calculating several key metrics. (use yfinance library)

1. Using the yfinance library, download the daily stock data for the ticker "TSLA" for the first half of 2023 (from "2023-01-01" to "2023-06-30"). Store the closing prices in a list. (4 Points)
2. First, use a for loop to calculate the daily percentage returns from the list of closing prices. Then, write a function that takes this list of returns and calculates its standard deviation. The formula for standard deviation is $\sigma = \sqrt{\frac{\sum(x_1-\mu)^2}{N}}$, where μ is the mean of the returns. (4 Points)
3. Using a while loop, iterate through your list of daily returns. Count the number of days where the absolute daily return was greater than 4% (0.04). (4 Points)
4. Loop through the daily returns and use if statements to keep track of the highest single-day gain and the largest single-day loss during the period. (4 Points)
5. Create a formatted string that reports the total number of trading days analyzed, the calculated volatility (formatted to 4 decimal places), the number of "High Volatility" days, and the best and worst day's performance (formatted as percentages with 2 decimal places). (4 Points)

Note - If you do 2nd sub question without using any external statistics libraries like numpy or pandas, you get 5 bonus points.

Question 3 — 20 Points

You are given a raw, multi-line string representing a log of stock trades. The formatting has some inconsistencies like extra whitespace and varied casing. Your task is to write a script to parse this log, calculate final share holdings for each stock, and generate a summary report.

Dataset -

```
trade_log = """

#Trade Log for 2025-10-03

  aapl, BUY, 100, 150.25

msft, BUY, 50, 300.50

  aapl, SELL, 25, 155.75


# A quick note on the next trade

GOOGL, buy, 10, 1350.00

msft, SELL, 50, 305.20

"""
```

1. Write a function parse_trade(log_line) that takes one line from the log.
   - It should check if the line is a valid trade. If the line is empty or starts with a #, the function should return None.
   - It should parse the line (e.g., " aapl, BUY, 100, 150.25 ") by splitting it by the comma.
   - It must clean the extracted data: The function should strip() whitespace from each part and convert the Ticker and Action to uppercase. The Quantity should be an integer and the Price a float.
   - The function should return a tuple with the cleaned data, like ('AAPL', 'BUY', 100, 150.25), or None if the line is not a valid trade.
2. Use a for loop to iterate through each line of the trade_log. Call your parse_trade() function on each line and create a new list that contains only the valid trade tuples, skipping any lines that return None.
3. Initialize an empty dictionary called portfolio. Use a for loop to iterate through your list of valid trade tuples. For each trade:
   - Use if/elif to check if the action is 'BUY' or 'SELL'.
   - Update the share count for the corresponding ticker in the portfolio dictionary. If a ticker is not yet in the dictionary, you should add it. (For example, after the first trade, your portfolio should be {'AAPL': 100}).
4. Create a new dictionary called trade_counts. Loop through the valid trades again and count how many trades (both buy and sell) occurred for each unique ticker.
5. Create a final, formatted multi-line f-string that summarizes the results. It should list each ticker, its final share holdings, and the total number of trades for that ticker.

Question 4. 20 Points

1. Use yfinance to download the daily 'Volume' data for the ticker "GOOGL" for the month of November 2023 (from "2023-11-01" to "2023-11-30"). Store the volumes in a list.
2. Calculate the total trading volume for the month. You must use a for loop to iterate through the list and add each day's volume to a running total. Do not use the built-in sum() function for this part.
3. A "high volume" day is any day with more than 30 million shares traded. Use a for loop to find the *index* of the first high volume day in your list. Use a break statement to exit the loop as soon as you find it. If no such day exists, the value should be -1.
4. Imagine you have a block of 150 million shares to sell. Use a while loop to "sell" shares by subtracting each day's trading volume from your total block. Count how many trading days it takes for your block of shares to be fully sold (i.e., for the total to go below zero).
5. Create a formatted string that reports the total volume, the index of the first high-volume day, and the number of days it took to sell the block of shares.

Question 5 — 20 Points

Dataset -

```
expenses = {
"Rent": 1500,
"Groceries": 450,
"Utilities": 120,
"Transport": 150
}
```

1. Write a function categorize_expense(amount) that returns "High" if the amount is greater than $300, "Medium" if it's between $100 and $300 (inclusive), and "Low" otherwise.
2. Use a for loop to iterate through the expenses dictionary. For each item, call your categorize_expensefunction and print a formatted string, like " - Rent: $1500 (High)".
3. Write a while True loop that prompts the user: "Add an expense (e.g., 'Coffee, 25') or type 'done': ".
   - If the user types "done", the loop should break.
   - If the user enters a string without a comma, print an error message and continue.
   - Otherwise, parse the input to get the item and amount, and add it to the expenses dictionary.
4. Write a function calculate_total(expense_dict) that takes a dictionary and uses a for loop to calculate and return the sum of all expense amounts.

5. After the loop finishes, call your calculate_total function on the updated expenses dictionary and print a final summary showing the total number of expense items and the total monthly cost.